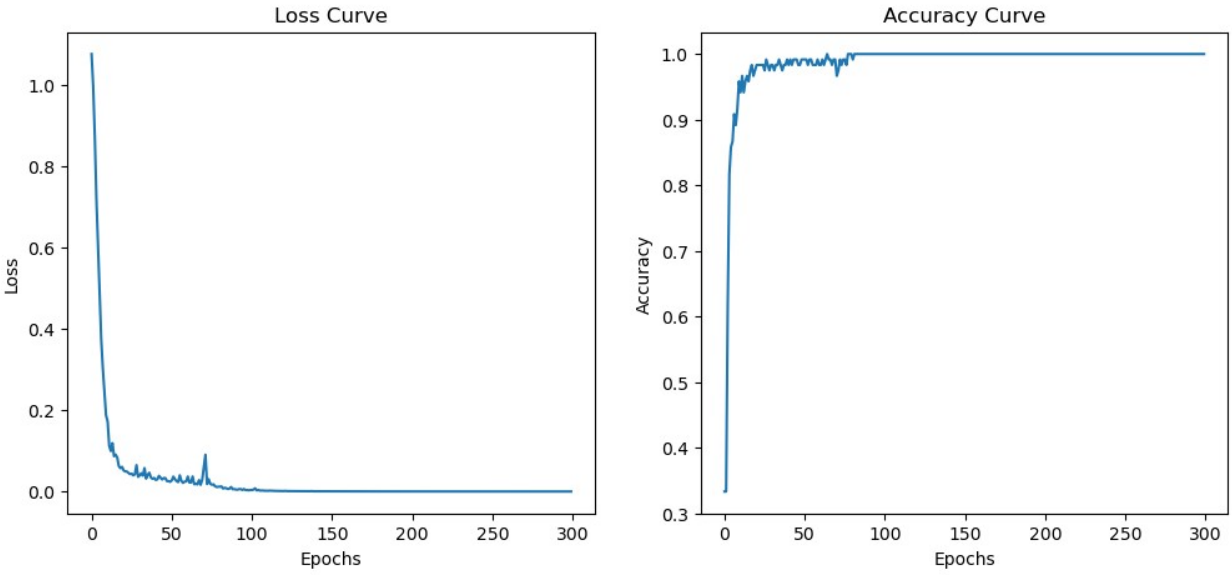
```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, accuracy_score
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("C:/Users/cools/Desktop/ML ass/iris.csv")
X = data.drop('species', axis=1).values
y = data['species'].values

# Encode labels
encoder = LabelEncoder()
y = encoder.fit_transform(y)

# Standardize features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Convert to tensors
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

train_dataset = TensorDataset(X_train, y_train)
test_dataset = TensorDataset(X_test, y_test)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

# Define the MLP model
class MLP(nn.Module):
    def __init__(self, activation_fn=nn.ReLU):
        super(MLP, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(4, 64),
            activation_fn(),
            nn.Linear(64, 128),
            activation_fn(),
            nn.Linear(128, 64),
            activation_fn(),
```

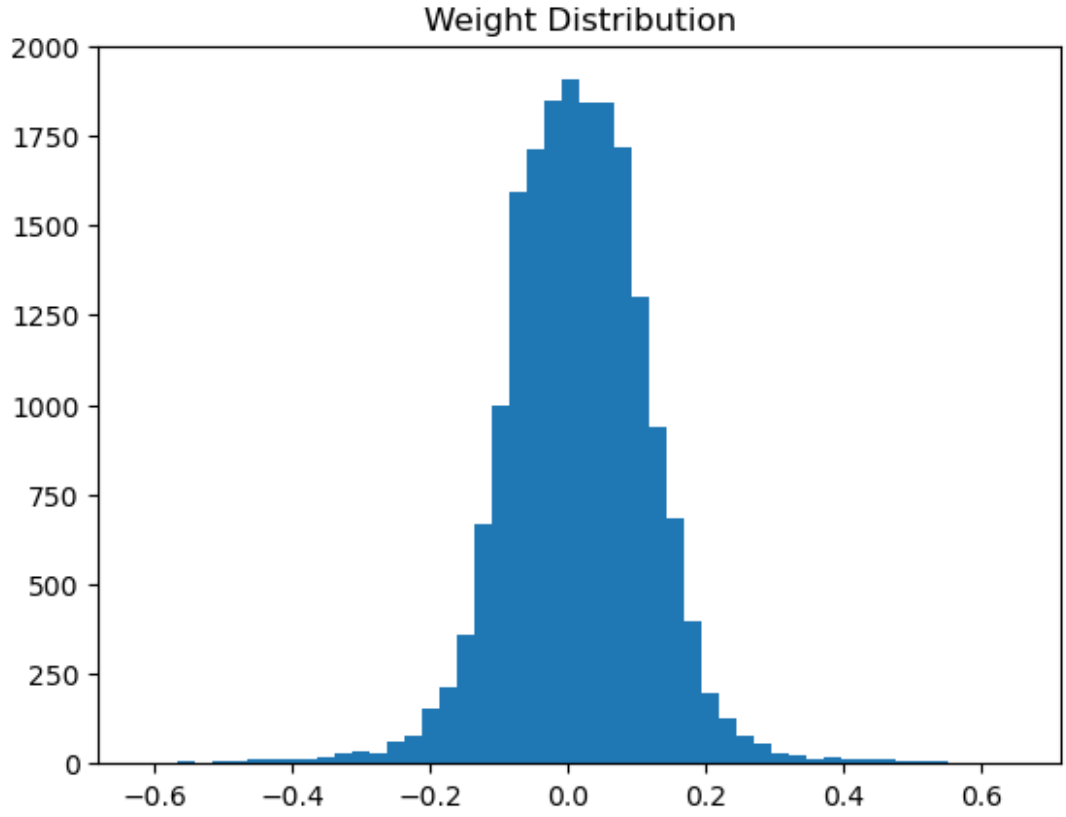
```
            nn.Linear(64, 32),
            activation_fn(),
            nn.Linear(32, 3)
        )

    def forward(self, x):
        return self.model(x)

# Training function
def train_model(activation_fn, loss_fn, optimizer_fn, epochs=300,
lr=0.001):
    model = MLP(activation_fn)
    optimizer = optimizer_fn(model.parameters(), lr=lr)
    criterion = loss_fn

    losses = []
    accuracies = []

    for epoch in range(epochs):
        model.train()
        epoch_loss = 0
        correct = 0

        for batch_X, batch_y in train_loader:
            optimizer.zero_grad()
            outputs = model(batch_X)
            if isinstance(criterion, nn.MSELoss):
                batch_y_onehot = nn.functional.one_hot(batch_y,
num_classes=3).float()
                loss = criterion(outputs, batch_y_onehot)
            else:
                loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()

            preds = outputs.argmax(dim=1)
            correct += (preds == batch_y).sum().item()

        losses.append(epoch_loss / len(train_loader))
        accuracies.append(correct / len(train_dataset))

    # Evaluate on test set
    model.eval()
    y_pred = []
    with torch.no_grad():
        for batch_X, _ in test_loader:
            outputs = model(batch_X)
            preds = outputs.argmax(dim=1)
            y_pred.extend(preds.cpu().numpy())
```

```
    print(classification_report(y_test, y_pred,
target_names=encoder.classes_))

    # Plot loss and accuracy
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,5))
    ax1.plot(losses)
    ax1.set_title('Loss Curve')
    ax1.set_xlabel('Epochs')
    ax1.set_ylabel('Loss')

    ax2.plot(accuracies)
    ax2.set_title('Accuracy Curve')
    ax2.set_xlabel('Epochs')
    ax2.set_ylabel('Accuracy')

    plt.show()

    # Plot weight distribution
    weights = []
    for param in model.parameters():
        if param.requires_grad:
            weights.extend(param.detach().cpu().numpy().flatten())

    plt.hist(weights, bins=50)
    plt.title('Weight Distribution')
    plt.show()

# Example of running different experiments
# You can call train_model() with different settings:
train_model(activation_fn=nn.ReLU, loss_fn=nn.CrossEntropyLoss(),
optimizer_fn=optim.Adam, epochs=300, lr=0.001)
```

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```


Loss Curve
Loss
Epochs
0.0
0.2
0.4
0.6
0.8
1.0
0
50
100
150
200
250
300

Accuracy Curve
Accuracy
Epochs
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1.0
0
50
100
150
200
250
300


Weight Distribution
0
250
500
750
1000
1250
1500
1750
2000
−0.6
−0.4
−0.2
0.0
0.2
0.4
0.6